

```
        oBank.closeAccount()

    elif action == 'd':
        oBank.deposit()

    elif action == 'o':
        oBank.openAccount()

    elif action == 's':
        oBank.show()

    elif action == 'q':
        break

    elif action == 'w':
        oBank.withdraw()

    else:
        print('Sorry, that was not a valid action. Please try again.')

print('Done')
```

Listing 4-8: The main code that creates a `Bank` object and makes calls to it

Notice how the code in [Listing 4-8](#) presents the top-level menuing system. It asks the user for an action ❶, then calls an appropriate method in the `Bank` object to do the work ❷. You could easily extend the `Bank` object to handle some additional queries, like asking for the bank’s hours, or address, or phone number. That data could simply be kept as additional instance variables inside the `Bank` object. The `Bank` would answer those questions without needing to communicate with any `Account` object.

When a close request is made ❸, the main code calls the `closeAccount()` method of the `Bank` object to close the account. The `Bank` object removes the specific account from its dictionary of accounts using a line like this:

```
del self.accountsDict[userAccountNumber]
```

Recall that our definition of an object is data, plus code that acts on that data, over time. The ability to delete an object demonstrates the third part of our definition of an object. We can create an object (in this case an `Account`